

Document de Mise en Production

Application « Vide Grenier en Ligne »

Auteur	Ilyan Tombette
Projet	Bloc 5 – Déployer et maintenir une application informatique
Dépôt Git	github.com/Tymate9/vide-grenier-en-ligne
Public visé	Développeur / administrateur système (collègue)
Objet	Procédure complète de remontée du système

Sommaire

- 1. Présentation de l'application
- 2. Architecture applicative
- 3. Prérequis de déploiement
- 4. Chronologie de mise en production
- 5. Environnements disponibles
- 6. Tests et validation
- 7. Résolution des incidents connus

1. Présentation de l'application

« Vide Grenier en Ligne » est une application web de don d'objets d'occasion entre particuliers. Elle repose sur un framework PHP maison suivant le patron de conception MVC (Modèle-Vue-Contrôleur), avec un rendu des vues via le moteur de templates Twig.

Fonctionnalités principales

- Inscription / Connexion, avec une option « Se souvenir de moi » (cookie sécurisé)
- Publication et consultation d'annonces, avec upload de photo obligatoire
- Formulaire de contact entre un acheteur et le vendeur d'une annonce, envoyé via SMTP (Mailtrap en environnement de test)

2. Architecture applicative

2.1 Structure du projet (MVC)

L'application est organisée selon une architecture Modèle-Vue-Contrôleur :

```
public/          -> Point d'entree HTTP (index.php, .htaccess)
App/
  Config.php     -> Constantes de configuration (base de donnees)
  Controllers/   -> Logique metier (User, Product, Home...)
  Models/        -> Acces a la base de donnees via PDO
  Views/         -> Templates Twig (.html)
  Utility/       -> Services transverses (Upload, Hash, Validator)
Core/
  Router.php     -> Routeur HTTP
  View.php       -> Wrapper Twig
  Model.php      -> Classe de base PDO
sql/import.sql   -> Script d'initialisation de la base
tests/          -> Tests unitaires et d'integration (PHPUnit)
Dockerfile      -> Image de developpement (php:8.2-apache, code en volume)
Dockerfile.deploy -> Image de deploiement (php:8.2-apache, code COPIE dans l'image)
apache-vhost.conf -> Configuration Apache (DocumentRoot -> public/)
entrypoint.sh   -> Genere la config msmtpl puis lance Apache
docker-compose.yml -> Developpement (port 8080)
docker-compose.recette.yml -> Recette (port 8081)
docker-compose.prod.yml -> Production (port 8083)
```

2.2 Stack technique

Composant	Technologie	Version
Langage back-end	PHP	8.2
Serveur web	Apache (mod_rewrite) intégré à l'image	2.4
Moteur de templates	Twig	^3.0
Base de données	MySQL	8.0
Accès BDD	PDO (pdo_mysql)	natif PHP
Envoi d'e-mails	mail() native PHP relayée par msmtpl vers Mailtrap	—
Tests	PHPUnit	^11
Conteneurisation	Docker + Docker Compose	—

2.3 Architecture des conteneurs Docker

Chaque environnement repose sur deux conteneurs, conformément à l'architecture demandée (un service web + une base de données) :

- Conteneur web : image php:8.2-apache. Apache et PHP tournent dans le même conteneur (module mod_php). En recette et production, le code est copié dans l'image au moment du build (image autonome, taguée vide-grenier-app:recette ou vide-grenier-app:prod).
- Conteneur base de données : image mysql:8.0. En production, les données sont stockées sur un volume nommé persistant (db_prod_data) qui survit aux redémarrages et créations du conteneur.

La configuration (hôte de base de données, identifiants, paramètres SMTP) est injectée via un fichier .env et des variables d'environnement, jamais en dur dans le code ni dans les images.

2.4 Flux d'une requête HTTP

```

Navigateur
-> Conteneur web (Apache, port 80 interne)
  -> public/index.php (front controller)
    -> Core/Router.php (routage)
      -> App/Controllers/... (logique metier)
        -> App/Models + Views (donnees + rendu Twig)
          -> Conteneur MySQL (via PDO)
  
```

Le fichier public/.htaccess réécrit toutes les URLs vers index.php (routeur), ce qui nécessite le module Apache mod_rewrite (activé dans l'image) et la directive AllowOverride All (définie dans apache-vhost.conf).

3. Prérequis de déploiement

Avant de lancer la mise en production, vérifier que la machine dispose de :

- Docker Engine (≥ 20.x) et le plugin Docker Compose (v2)
- Git, pour cloner le dépôt
- Les ports 8080, 8081 et 8083 libres sur l'hôte (développement, recette, production)
- Un accès internet (téléchargement des images Docker de base)

4. Chronologie de mise en production

La procédure ci-dessous permet de remonter l'environnement de production à partir de zéro. Les commandes sont à exécuter sous Windows (PowerShell), d'où l'emploi de docker.exe.

Étape 1 — Cloner le dépôt et se placer sur la branche de production

```
git clone https://github.com/Tymate9/vidé-grenier-en-ligne.git
cd vidé-grenier-en-ligne
git checkout main
```

```
PS C:\Windows\system32> git clone https://github.com/Tymate9/vidé-grenier-en-ligne.git test-clone
Cloning into 'test-clone'...
remote: Enumerating objects: 296, done.
remote: Counting objects: 100% (296/296), done.
remote: Compressing objects: 100% (228/228), done.
remote: Total 296 (delta 94), reused 259 (delta 57), pack-reused 0 (from 0)
Receiving objects: 100% (296/296), 7.38 MiB | 2.58 MiB/s, done.
Resolving deltas: 100% (94/94), done.
PS C:\Windows\system32> cd test-clone
>>
PS C:\Windows\system32\test-clone> git branch -a
>>
* main
remotes/origin/HEAD -> origin/main
remotes/origin/dev
remotes/origin/main
remotes/origin/recette
```

Clonage du dépôt et vérification des branches disponibles.

Étape 2 — Construire l'image de production

L'image embarque le code et toutes ses dépendances, de façon autonome :

```
docker.exe build -f Dockerfile.deploy -t vidé-grenier-app:prod .
```

```
PS C:\Users\FX507\Downloads\vidé-grenier-en-ligne-master\vidé-grenier-en-ligne-master> docker.exe build -f Dockerfile.deploy -t vidé-grenier-app:prod .
[+] Building 2.3s (19/19) FINISHED
=> [internal] load build definition from Dockerfile.deploy
=> => transferring dockerfile: 851B
=> [internal] load metadata for docker.io/library/composer:latest
=> [internal] load metadata for docker.io/library/php:8.2-apache
=> [internal] load .dockerignore
=> => transferring context: 588B
=> [internal] load build context
=> => transferring context: 5.53kB
=> FROM docker.io/library/composer:latest@sha256:e8fdff913656c23e90ebbe0d7c55ab078c2aefdbb53ff79a73af5cc0921d5b81
=> => resolve docker.io/library/composer:latest@sha256:e8fdff913656c23e90ebbe0d7c55ab078c2aefdbb53ff79a73af5cc0921d5b81
=> [stage-0 1/12] FROM docker.io/library/php:8.2-apache@sha256:1628cad804f867206938d9f693bae5d05a2d7db6c58752ff0c3acc1c2b5eebca
=> => resolve docker.io/library/php:8.2-apache@sha256:1628cad804f867206938d9f693bae5d05a2d7db6c58752ff0c3acc1c2b5eebca
=> CACHED [stage-0 2/12] RUN apt-get update && apt-get install -y git unzip libzip-dev msmt msmt-mta && docker-php-ext-install zip pd
=> CACHED [stage-0 3/12] RUN echo 'sendmail_path = "/usr/bin/msmtp -t" > /usr/local/etc/php/conf.d/mail.ini
=> CACHED [stage-0 4/12] RUN a2enmod rewrite
=> CACHED [stage-0 5/12] COPY --from=composer:latest /usr/bin/composer /usr/bin/composer
=> CACHED [stage-0 6/12] COPY apache-vhost.conf /etc/apache2/sites-available/000-default.conf
=> CACHED [stage-0 7/12] COPY entrypoint.sh /usr/local/bin/entrypoint.sh
=> CACHED [stage-0 8/12] RUN sed -i 's/\r$//' /usr/local/bin/entrypoint.sh && chmod +x /usr/local/bin/entrypoint.sh
=> CACHED [stage-0 9/12] WORKDIR /var/www/html
=> CACHED [stage-0 10/12] COPY . .
=> CACHED [stage-0 11/12] RUN composer install --no-dev --optimize-autoloader --no-interaction
=> CACHED [stage-0 12/12] RUN chown -R www-data:www-data public/storage
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:cf3598760f45eef596ae8e1b78d7006934c0b6ee1009cd39f0e7c6bd33f3ff3c
=> => exporting config sha256:4715165deed5b0ea940088ef8ac395132138963ccb3a37304b595da151c61b8
=> => exporting attestation manifest sha256:65a30e2cb90988d8fc4309d7bf8487ad5f7e2d4102d157ee61229d6844530d2
=> => exporting manifest list sha256:0ad529d169d8606a392dd5c69c50234964df740700592e58ec9bef13b0724390
=> => naming to docker.io/library/vidé-grenier-app:prod
=> => unpacking to docker.io/library/vidé-grenier-app:prod
```

Construction de l'image taguée vidé-grenier-app:prod (build terminé).

Étape 3 — Démarrer les conteneurs de production

```
docker.exe compose -f docker-compose.prod.yaml up -d --remove-orphans
```

Vérifier que les deux conteneurs sont actifs et que la base de données est saine :

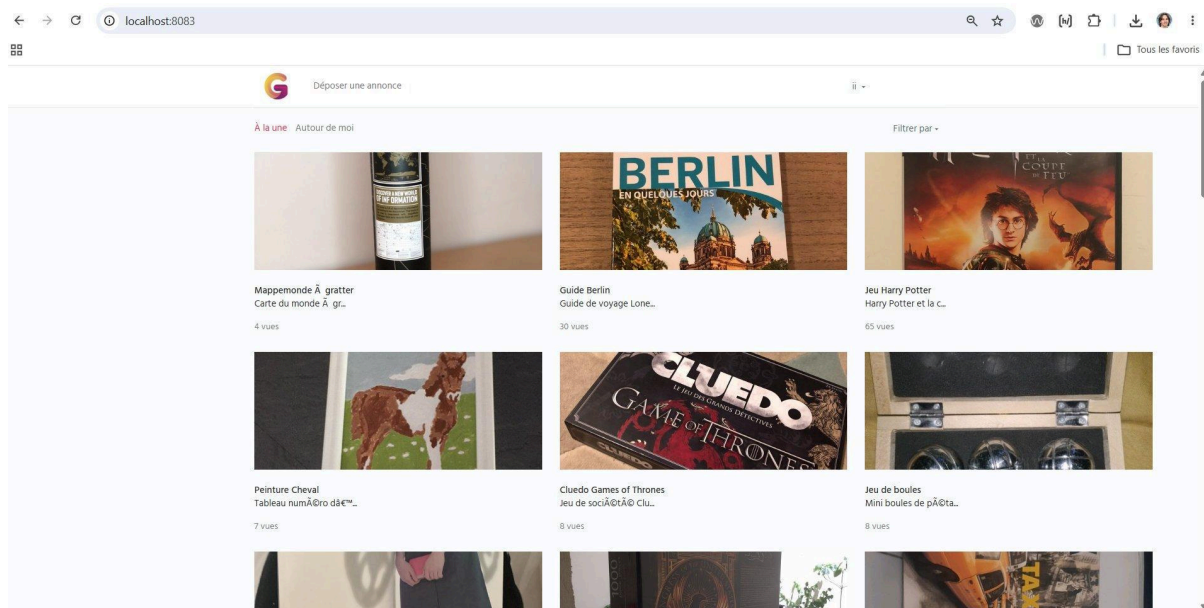
```
docker.exe compose -f docker-compose.prod.yaml ps
```

```
PS C:\Users\FX507\Downloads\vidé-grenier-en-ligne-master\vidé-grenier-en-ligne-master> docker.exe compose -f docker-compose.prod.yaml ps
NAME                IMAGE                COMMAND                  SERVICE   CREATED      STATUS      PORTS
vidé-grenier_db_prod mysql:8.0            "docker-entrypoint.s..." db        18 minutes ago Up 18 minutes (healthy) 3306/tcp, 3306/tcp
vidé-grenier_web_prod sha256:96b1e4a586f0924c16f5514cd17b701424a2d294bd8a2157e325fa77788b4ef "entrypoint.sh apach..." web       18 minutes ago Up 18 minutes 0.0.0.0:8083->80/tcp
```

Les conteneurs web_prod et db_prod sont « Up », db_prod est « healthy ».

Étape 4 — Vérification post-déploiement

Ouvrir <http://localhost:8083> dans un navigateur et vérifier le bon fonctionnement : page d'accueil, inscription, connexion, dépôt d'annonce, contact vendeur, déconnexion.



Page d'accueil de l'application sur <http://localhost:8083>, correctement stylée.

5. Environnements disponibles

Les trois environnements sont conçus pour coexister simultanément sur la même machine (ports et réseaux Docker distincts) :

Environnement	Fichier Compose	Port	Code dans l'image ?
Développement	docker-compose.yaml	8080	Non (monté en volume)
Recette	docker-compose.recette.yaml	8081	Oui (image taguée recette)
Production	docker-compose.prod.yaml	8083	Oui (image taguée prod)

Pour lancer les trois en même temps :

```
docker.exe compose up -d # dev (8080)
docker.exe compose -f docker-compose.recette.yaml up -d # recette (8081)
docker.exe compose -f docker-compose.prod.yaml up -d # production (8083)

PS C:\Windows\system32> docker.exe ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}\t{{.Status}}"
NAMES          IMAGE          PORTS          STATUS
vide_grenier_web_prod  96b1e4a586f0  0.0.0.0:8083->80/tcp  Up 28 minutes
vide_grenier_db_prod   mysql:8.0      3306/tcp, 33060/tcp  Up 28 minutes (healthy)
)
vide_grenier_web_recette vide-grenier-app:recette 0.0.0.0:8081->80/tcp  Up 35 minutes
vide_grenier_db_recette mysql:8.0      3306/tcp, 33060/tcp  Up 59 minutes (healthy)
)
vide_grenier_web_dev   vide-grenier-en-ligne-master-web 0.0.0.0:8080->80/tcp  Up 58 minutes
vide_grenier_db_dev    mysql:8.0      0.0.0.0:3306->3306/tcp, 33060/tcp  Up About an hour (healthy)
```

Les 6 conteneurs (dev, recette, production) actifs en même temps — preuve de la coexistence des environnements.

6. Tests et validation

L'application est couverte par des tests automatisés PHPUnit : tests unitaires (classe Hash, classe Validator) et tests d'intégration (modèle User contre la base de données). Ils servent de tests de non-régression.

Pour les exécuter sur l'environnement de développement :

```
docker.exe exec vide_grenier_web_dev vendor/bin/phpunit
```

Remarque : les images de recette et production sont construites avec `composer install --no-dev`, donc PHPUnit n'y est pas embarqué (image allégée et sécurisée pour la livraison).

```
PS C:\Users\FX507\Downloads\vide-grenier-en-ligne-master\vide-grenier-en-ligne-master> docker.exe exec vide_grenier_web_dev vendor/bin/phpunit
PHPUnit 11.5.55 by Sebastian Bergmann and contributors.

Runtime:       PHP 8.2.30
Configuration: /var/www/html/phpunit.xml

.....                                     12 / 12 (100%)

Time: 00:00.358, Memory: 8.00 MB

OK (12 tests, 67 assertions)
```

Exécution de PHPUnit : 12 tests, 67 assertions, tous au vert.

7. Résolution des incidents connus

Incidents rencontrés et corrigés durant le développement et le déploiement :

Incident	Cause / Correction
Class "Twig\Loader\Filesystemloader" not found (erreur 500 en recette)	Faute de casse dans Core/View.php. Invisible en dev (Windows insensible à la casse), fatale en image Linux. Corrigée.
Photo non obligatoire à la publication d'une annonce	Ajout d'une validation côté serveur + attribut required, et suppression d'un formulaire HTML imbriqué dans la vue.
Pas de connexion automatique après inscription	Appel de la fonction de connexion à la fin de l'inscription, puis redirection.
« Se souvenir de moi » non fonctionnel	Ajout d'une colonne remember_token en base + cookie HttpOnly contenant un jeton aléatoire, dont seul le hash est stocké en base.
Le contact vendeur ouvrait la boîte mail (mailto)	Remplacement du lien mailto par un vrai formulaire de contact, envoyé via msmtpt vers Mailtrap en environnement de test.
Feuilles de style (CSS) non chargées après conteneurisation	Passage d'une architecture nginx + php-fpm à un conteneur unique php:8.2-apache servant pages et fichiers statiques.
entrypoint.sh introuvable au démarrage (image Linux)	Fins de ligne Windows (CRLF) dans le script. Correction via sed -i 's/\r\$//' dans le Dockerfile.